Fiche de révision — PHP

Acquis couverts

- Coder en PHP avec une syntaxe correcte
- Manipuler les chaînes de caractères
- Utiliser les fonctions `mb_*` pour l'UTF-8
- Appliquer les bases de la POO
- Gérer les entrées utilisateur de façon sécurisée
- Déboguer avec `var_dump()`
- Connaître les principales superglobales
- Mettre en place un téléversement de fichier

1 PHP : principe

PHP est un langage de script exécuté **côté serveur**. Le serveur exécute le code PHP puis renvoie au navigateur le résultat généré.

```
<?php
echo "Bonjour !";
?>
```

Exécution en ligne de commande :

```
php test.php
```

2 Syntaxe de base

2.1 Variables et types

En PHP, une variable commence par `$`.

```
<?php
$nom = "Chlo";
$age = 20;
$prix = 12.5;
$ok = true;
?>
```

2.2 Conditions

```
<?php
if ($age >= 18) {
    echo "Majeur";
} else {
    echo "Mineur";
}
?>
```

2.3 Boucles

```
<?php
for ($i = 0; $i < 3; $i++) {
    echo $i . "<br>";
}
?>
```

2.4 Tableaux

```
<?php
$notes = [12, 15, 18];

$user = [
    "nom" => "Chlo",
    "age" => 20
];

echo $notes[0];
echo $user["nom"];
?>
```

2.5 Fonctions

```
<?php
function carre(int $x): int {
    return $x * $x;
}

echo carre(4);
?>
```

3 Chaînes de caractères

Une chaîne est du texte.

3.1 Quotes simples

Avec des quotes simples, PHP n'interprète pas les variables.

```
<?php
$nom = "Marie";
echo 'Bonjour $nom';
?>
```

Résultat : `Bonjour $nom`

3.2 Quotes doubles

Avec des quotes doubles, PHP interprète les variables.

```
<?php
$nom = "Marie";
echo "Bonjour $nom";
?>
```

Résultat : `Bonjour Marie`

3.3 Concaténation

On concatène avec le point .

```
<?php
$nom = "Marie";
echo "Bonjour " . $nom;
?>
```

3.4 Caractères échappés

```
<?php
echo "Ligne 1\nLigne 2";
echo "Il a dit : \"Bonjour\"";
?>
```

4 Heredoc et Nowdoc

4.1 Heredoc

Pratique pour écrire du texte sur plusieurs lignes avec interprétation des variables.

```
<?php
$nom = "Marie";

echo <<<TXT
Bonjour $nom
Bienvenue en PHP
TXT;
?>
```

4.2 Nowdoc

Même principe, mais sans interprétation.

```
<?php
$nom = "Marie";

echo <<<'TXT'
Bonjour $nom
Bienvenue en PHP
TXT;
?>
```

5 Encodage, ASCII et UTF-8

À retenir

- **ASCII** : ancien jeu de caractères limité
- **UTF-8** : encodage moderne, très utilisé sur le Web
- Certains caractères comme `é` ou les emojis prennent plusieurs octets

Exemple important :

```
<?php
echo strlen("");
?>
```

Le résultat peut être surprenant car `strlen()` compte les **octets**, pas forcément les caractères visibles.

5.1 Fonctions `mb_*`

Pour gérer correctement l'UTF-8, on utilise les fonctions multibytes.

```
<?php
echo mb_strlen("", "UTF-8");
?>
```

Autres fonctions utiles :

```
<?php
$texte = "lve";

echo mb_strtoupper($texte, "UTF-8");
echo mb_strtolower($texte, "UTF-8");
echo mb_substr($texte, 0, 2, "UTF-8");
?>
```

6 Typage en PHP

PHP est un langage à typage dynamique.

```
<?php
$a = "10";
$b = 10;

var_dump($a == $b); // true
var_dump($a === $b); // false
?>
```

Différence importante

`==` compare les valeurs après conversion éventuelle.
`===` compare la valeur **et** le type.

7 Programmation orientée objet (POO)

7.1 Classe et objet

```
<?php
class User {
    public $name;
}

$user = new User();
$user->name = "Marie";

echo $user->name;
?>
```

7.2 Constructeur

```
<?php
class User {
    public $name;
```

```
    public function __construct($name) {
        $this->name = $name;
    }
}

$user = new User("Marie");
echo $user->name;
?>
```

7.3 Encapsulation

```
<?php
class User {
    private $password;

    public function setPassword($password) {
        $this->password = $password;
    }

    public function getPassword() {
        return $this->password;
    }
}
?>
```

7.4 Héritage

```
<?php
class User {
    public $name;
}

class Admin extends User {
    public $role = "admin";
}

$admin = new Admin();
$admin->name = "Chlo";

echo $admin->name;
echo $admin->role;
?>
```

8 Entrées utilisateur et sécurité

Il ne faut jamais faire confiance aux données envoyées par l'utilisateur.

8.1 Avec $_POST

```
<?php
$nom = $_POST["nom"] ?? "";
?>
```

8.2 Échappement HTML

Pour éviter des injections HTML ou XSS :

```
<?php
$nom = htmlspecialchars($_POST["nom"] ?? "", ENT_QUOTES, "UTF-8");
echo $nom;
?>
```

Bon réflexe

Toujours valider, filtrer et échapper les entrées utilisateur avant affichage ou traitement.

9 Débogage avec `var_dump()`

`var_dump()` affiche le type et la valeur d'une variable.

```
<?php
$age = 20;
var_dump($age);

$tableau = ["nom" => "Chlo", "age" => 20];
var_dump($tableau);
?>
```

Autres outils utiles :

```
<?php
print_r($tableau);
die("Arrt du script");
?>
```

10 Superglobales importantes

Les superglobales sont accessibles partout dans le script.

- `$_GET` : données dans l'URL
- `$_POST` : données envoyées par formulaire
- `$_SERVER` : infos sur le serveur et la requête
- `$_FILES` : fichiers téléversés
- `$_COOKIE` : cookies
- `$_SESSION` : session utilisateur
- `$_ENV` : variables d'environnement
- `$_REQUEST` : mélange de GET/POST/COOKIE

Exemples :

```
<?php
echo $_GET["page"] ?? 1;
echo $_SERVER["REQUEST_METHOD"];
?>
```

11 Pagination

La pagination permet d'afficher des résultats sur plusieurs pages.

```
<?php
$elementsParPage = 10;
$page = isset($_GET["page"]) ? (int)$_GET["page"] : 1;
$totalElements = 50;
```

```
$totalPages = ceil($totalElements / $elementsParPage);
$offset = ($page - 1) * $elementsParPage;
?>
```

Exemple avec un tableau :

```
<?php
$articles = range(1, 50);
$articlesPage = array_slice($articles, $offset, $elementsParPage);

foreach ($articlesPage as $article) {
    echo "Article $article<br>";
}
?>
```

Liens de pagination :

```
<?php
for ($i = 1; $i <= $totalPages; $i++) {
    echo '<a href="?page=' . $i . '">' . $i . '</a> ';
}
?>
```

12 Téléversement de fichiers

12.1 Formulaire HTML

```
<form action="upload.php" method="POST" enctype="multipart/form-data">
    <label>Votre nom :</label>
    <input type="text" name="nom" required>

    <label>Votre CV :</label>
    <input type="file" name="cv" required>

    <button type="submit">Envoyer</button>
</form>
```

12.2 Structure de $_FILES

```
<?php
$_FILES["cv"]["name"];
$_FILES["cv"]["type"];
$_FILES["cv"]["tmp_name"];
$_FILES["cv"]["error"];
$_FILES["cv"]["size"];
?>
```

12.3 Traitement du fichier

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nom = htmlspecialchars($_POST["nom"] ?? "", ENT_QUOTES, "UTF-8");
    $cv = $_FILES["cv"];

    if ($cv["error"] === 0) {
        $tailleMax = 2 * 1024 * 1024; // 2 MB
```

```
        if ($cv["size"] <= $tailleMax) {
            $extension = pathinfo($cv["name"], PATHINFO_EXTENSION);

            if ($extension === "pdf") {
                $dossier = "uploads/";
                $nouveauNom = uniqid() . ".pdf";

                move_uploaded_file($cv["tmp_name"], $dossier . $nouveauNom);

                echo "CV envoy avec succs";
            } else {
                echo "Format invalide";
            }
        } else {
            echo "Fichier trop volumineux";
        }
    } else {
        echo "Erreur lors de l'envoi";
    }
}
?>
```

12.4 Étapes importantes

- Vérifier que le formulaire est soumis en POST
- Vérifier `$_FILES["..."]["error"]`
- Vérifier la taille du fichier
- Vérifier l'extension et le type MIME
- Générer un nom unique
- Déplacer le fichier avec `move_uploaded_file()`

12.5 Sécurité upload

Très important

- Ne jamais faire confiance au nom d'origine
- Vérifier l'extension **et** le type MIME
- Stocker les fichiers hors du dossier public si possible
- Limiter la taille maximale
- Désactiver l'exécution dans le dossier `uploads`

Structure possible :

```
project/
    index.php
    upload.php
    uploads/
```

13 Résumé express

À maîtriser

- Syntaxe PHP : variables, conditions, boucles, tableaux, fonctions
- Chaînes : simples, doubles, concaténation, heredoc, nowdoc
- UTF-8 et fonctions `mb_*`
- POO : classe, objet, constructeur, héritage, encapsulation
- Sécurité des entrées : `htmlspecialchars()`
- Débogage : `var_dump()`
- Superglobales : `$_GET`, `$_POST`, `$_SERVER`, `$_FILES`, etc.
- Pagination
- Téléversement de fichier sécurisé